Integración de Sistemas · Unidad 3

Encargo integrador — FORMA M

Dominio: Arriendo de equipos — arriendo de un equipo

Sistema distribuido resiliente: exposición (Gateway/BFF) · transacción distribuida (Saga) · resiliencia (Circuit Breaker + Bulkhead) con evidencia ante fallas.

1. Presentación y objetivo

Este encargo cierra la Unidad 3 del curso. No es solo una tarea de programación: es un ejercicio de ingeniería donde el equipo integra varios servicios en un sistema distribuido, lo expone con criterio, lo hace sobrevivir a fallas, y —sobre todo— demuestra con evidencia que sus decisiones de diseño funcionan cuando algo se cae.

La materia disponible es todo lo visto en la Unidad 3 (patrones):

- Exposición: API Gateway (enrutamiento, autenticación, rate limiting) y BFF (agregación por cliente).
- Transacciones distribuidas: Two-Phase Commit y Saga (coreografía y orquestación), con compensaciones.
- Resiliencia: timeout, Circuit Breaker, Bulkhead, retry con backoff, degradación elegante.
- Observabilidad: logs/métricas para evidenciar el comportamiento ante fallas.

> *Lo que se evalúa no es solo que el sistema funcione, sino que las decisiones estén tomadas con criterio y que se demuestre, con evidencia, que los patrones contienen la falla. Una solución modesta bien justificada y bien evidenciada vale más que una ambiciosa que el equipo no sabe explicar ni demostrar.*

Modalidad

- Grupos de hasta 4 integrantes. Grupos más pequeños pueden acordar un alcance reducido con el docente.
- Todo el trabajo se registra en un repositorio Git con commits identificables por integrante.
- Todo el sistema debe levantarse con un único `docker compose up`.
- El encargo culmina en una defensa oral donde se pregunta por las decisiones y por el código.

2. El problema a resolver

2.1 Contexto (Forma M — Arriendo de equipos)

Una empresa arrienda equipos coordinando Disponibilidad (reserva el equipo para un periodo), Pagos (cobra el arriendo y la garantía por una pasarela externa) y Entrega (agenda la entrega del equipo). Si se cobra y no se entrega, o al revés, queda un descuadre. Piden integrar el arriendo de forma transaccional y resiliente.

2.2 Los servicios del dominio

- **Disponibilidad:** reserva y libera el equipo por periodo.
- **Pagos:** cobra y reembolsa arriendo y garantía (pasarela externa).

- **Entrega:** agenda y cancela la entrega del equipo.

2.3 La operación central: «Arrendar un equipo»

Esta operación cruza los tres servicios y es la que se implementa como Saga. Sus pasos, con la compensación sugerida para cada uno:

- **Paso 1:** Reservar el equipo (servicio Disponibilidad). Compensación: Liberar el equipo.
- **Paso 2:** Cobrar arriendo y garantía (servicio Pagos). Compensación: Reembolsar.
- **Paso 3:** Agendar la entrega (servicio Entrega). Compensación: Cancelar la entrega.

2.4 Reglas de negocio mínimas

- Un equipo no puede arrendarse a dos clientes en periodos superpuestos.
- No se agenda la entrega sin cobro exitoso; reintentar no cobra dos veces (idempotencia).

3. Lo que se pide construir (requisitos técnicos)

El equipo debe construir un sistema de microservicios dockerizado que exponga la operación central de su dominio, la implemente como una transacción distribuida con Saga, y la endurezca con patrones de resiliencia, demostrando su comportamiento ante fallas inducidas. Como mínimo:

3.1 Arquitectura mínima

- Los tres servicios de dominio de su forma, cada uno con su propia base de datos (o almacén).
- Un API Gateway como único punto de entrada al sistema.
- Al menos un BFF (web o móvil).
- Todo en un `docker-compose.yml`; solo el gateway (y/o el BFF) se expone al exterior.

3.2 Exposición — Gateway + BFF

- **Enrutamiento:** El gateway enruta hacia los servicios internos; estos no se exponen directamente.
- **Autenticación central:** El gateway valida un token en la entrada (los servicios confían en que ya pasó por la puerta).
- **Rate limiting:** El gateway limita las peticiones por cliente (protege de picos y abuso).
- **Agregación (BFF):** Al menos un BFF compone, en una sola respuesta a medida de un cliente, datos de 2 o más servicios.

3.3 Transacción distribuida — Saga

- La operación central se implementa como Saga. El equipo elige coreografía u orquestación y justifica por qué.
- Cada paso tiene su compensación. Se debe demostrar el camino feliz y, ante un fallo, la compensación en orden inverso de los pasos ya confirmados.
- Los pasos y compensaciones son idempotentes: un comando o evento repetido no causa daño (doble cobro, doble reserva).
- El informe debe justificar por qué se usó Saga y no Two-Phase Commit en este caso.

3.4 Resiliencia

- **Timeout:** Toda llamada entre servicios tiene un límite de espera.

- **Circuit Breaker:** Al menos sobre la dependencia frágil del dominio, con un fallback (degradación elegante) cuando está abierto.
- **Bulkhead:** Aislamiento de recursos (pool o límite de concurrencia) para que una dependencia lenta no agote los recursos del resto.

El corazón de la nota: inducir fallas en la dependencia frágil (lentitud, caída, pico) y presentar EVIDENCIA de que el sistema las contiene —el circuito abre y responde rápido, el bulkhead rechaza el exceso, y el resto del sistema sigue funcionando— en vez de caerse en cascada.

4. Escenarios de falla obligatorios a demostrar

La defensa debe mostrar, en vivo, al menos estos tres escenarios sobre su dominio. Para cada uno: provóquenlo, muestren qué hace el sistema, y presenten la evidencia.

1. **Éxito:** El camino feliz completo: la operación «arrendar un equipo» se completa y los tres pasos quedan firmes.
2. **Fallo con compensación:** Hagan fallar el ÚLTIMO paso de la saga: comprueben que se compensan los pasos anteriores en orden inverso y que no queda estado inconsistente (ni cobro sin operación, ni reserva sin confirmar).
3. **Falla en cascada contenida:** Pongan lenta y luego caída a la dependencia frágil (Pagos (pasarela externa): lenta o caída.) Demuestren con evidencia que el Circuit Breaker abre y responde rápido (fail-fast + fallback), y que el Bulkhead evita que esa lentitud agote los recursos del resto del sistema.

5. Entregables

- **Repositorio Git:** Levanta todo el sistema con `docker compose up`. Incluye un README con los pasos y cómo provocar cada escenario de falla.
- **Informe:** Máximo 8 páginas. Decisiones de arquitectura justificadas, diagramas del flujo feliz y de las fallas, y la evidencia (capturas de métricas/logs del antes y después).
- **Video:** Máximo 8 minutos. Demo en vivo: camino feliz, inducción de las fallas y lectura de la evidencia. Todos los integrantes participan.

6. Rúbrica de evaluación (competencias ABET)

La nota se construye sobre tres competencias. La competencia 6 (experimentación y análisis) pesa más: el núcleo de este encargo es inducir fallas y demostrar, con evidencia, que los patrones contienen el daño.

Competencia	Peso	Qué se evalúa
ABET 1 — Resolver problemas de ingeniería (implementación correcta)	30%	• Exposición: el gateway enruta, autentica y limita; el BFF agrega ≥2 servicios en una respuesta. • Saga: la operación central se completa (camino feliz) y, ante fallas, compensa correctamente en orden

		inverso. • Resiliencia: timeout, Circuit Breaker (con fallback) y Bulkhead funcionando sobre la dependencia frágil.
ABET 2 — Diseño con criterio (decisiones justificadas)	30%	• Justifica coreografía vs. orquestación y por qué Saga y no 2PC en este caso. • Diagramas correctos del flujo feliz y de las fallas. • Idempotencia y separación de responsabilidades (gateway sin lógica de negocio; BFF por cliente).
ABET 6 — Experimentación y análisis (evidencia ante fallas)	40%	• Diseña escenarios de falla claros y reproducibles (lentitud, caída, pico de carga). • Presenta evidencia del antes/después (métricas, logs, estado del circuito, rechazos del bulkhead). • Interpreta la evidencia: explica qué muestra y qué habría pasado SIN el patrón.

Escala de niveles (aplicada a cada competencia): Logrado (cumple y lo justifica) · Parcial (funciona pero con vacíos o sin justificar) · Incipiente (ausente o incorrecto). La defensa oral puede subir o bajar la nota: se pregunta por cualquier decisión y por cualquier línea del código.